

SMOLSolver: Lightweight Generator – Verifier Framework for Math Problem Solving

Akshat Singh, Zhuwei Xu, Shiyue Zhang, Raj Trikha, Nishant Sharma

New York University

{as20255, zx2188, sz5331, rt2932, ns6287}@nyu.edu

Abstract

We present SMOLSolver, a modular framework for math problem solving on GSM8K using small language models. Initially motivated by step-level iterative refinement, our work pivoted when closed-loop feedback with a TinyLLaMA verifier yielded no improvement. Instead, we reframed verification as a *post-hoc selection* problem. Our primary system generates 15 candidate solutions per problem, then applies consensus-aware aggregation using either a RoBERTa-based binary classifier (Verifier) or regression ranker (Ranker). We find that Majority Voting remains a formidable baseline, while ranker-weighted aggregation offers competitive performance. Crucially, we discover that generator behavior is highly heterogeneous: solutions naturally cluster into *high*, *medium*, and *low* consensus groups. In a pilot using an external math model (Qwen2.5-Math), we demonstrate that *adaptive expansion*—increasing sample count only for low-consensus problems—achieves favorable accuracy-compute tradeoffs. Our work underscores that for small models, knowing *when* to trust or expand may be more valuable than attempting to correct.

1 Introduction

Large language models achieve impressive performance on mathematical reasoning tasks like GSM8K, but their computational cost limits deployment in resource-constrained settings. Recent work shows that small language models (SLMs), when fine-tuned with parameter-efficient methods, can approach competitive accuracy. Inspired by this, we set out to build **SMOLSolver**: a lightweight, modular system combining a Generator and Verifier for step-by-step math problem solving.

Our original hypothesis was that a dedicated verifier could provide step-level feedback to iteratively refine the generator’s output—mirroring process supervision paradigms. However, integrating a

TinyLLaMA-based step verifier into a feedback loop **did not yield improvements** (Finding 1). In fact, it often degraded performance. This aligns with recent findings by Zhang et al. (2024), who report that small models struggle to effectively self-correct without stronger external feedback. We observed that the "verify-then-revise" loop often introduced new errors rather than fixing existing ones.

Consequently, we pivoted to a **post-hoc answer selection** paradigm. This approach acknowledges the *generator-validator gap* (Li et al., 2024; Rodriguez et al., 2025), where models often fail to consistently validate their own generations. SMOLSolver mitigates this by generating 15 candidate solutions and applying consensus-aware aggregation. Our system consists of:

- A LoRA-finetuned **Phi-2 Generator**.
- A RoBERTa-large **Verifier** (binary classifier).
- A RoBERTa-large **Ranker** (regression model).

Using these, we evaluate aggregation strategies across consensus groups. We find that while Majority Voting is robust, ranker-weighted aggregation offers competitive performance in medium-consensus regimes. We further explore adaptive expansion (Finding 4), a direction supported by recent "deep thinking" strategies like **rStar-Math** (Guan et al., 2025), showing that selective resampling based on consensus can improve efficiency.

Our contributions are: (1) We show step-level verifier correction fails for SLMs, corroborating Zhang et al. (2024). (2) We analyze Pass@k scaling, revealing the stochastic error nature of SLMs. (3) We introduce consensus grouping to characterize generator stability. (4) We show Ranker-weighted selection excels in medium/low-consensus settings. (5) We demonstrate a proof-of-concept adaptive sampling strategy.

2 Related Work

Small Models for Math While large models like GPT-4 dominate reasoning benchmarks, recent efforts have focused on empowering small language models. **Orca-Math** (Mitra et al., 2024) demonstrated that SLMs can achieve high performance on GSM8K through training on high-quality synthetic data, removing the need for massive scale. Our work builds on this by focusing on inference-time efficiency for such models.

Process vs. Outcome Supervision Verification typically operates at the *outcome* or *process* level. While large models benefit from "verify-then-revise" loops, Zhang et al. (2024) show that this self-correction capacity is largely absent in small models, which lack the robust internal verification logic required to critique their own outputs. Li et al. (2024) further quantify this as a consistency failure, where models cannot reliably validate statements they themselves generate.

Self-Consistency and Selection To address these limitations, we rely on **post-hoc selection**. This connects to the work of Rodriguez et al. (2025), who propose ranking-based methods to bridge the generator-validator gap. Recently, Guan et al. (2025) introduced **rStar-Math**, which uses deep search and self-evolved reasoning to boost SLM performance. Similarly, our adaptive expansion strategy leverages consensus signals to allocate compute dynamically, avoiding the pitfalls of fragile self-correction loops in favor of robust selection.

3 Method

3.1 System Overview

SMOLSolver’s modular pipeline (Figure 1) is built around a generator, followed by flexible scoring modules: a verifier (V), a ranker (R), or both, enabling different operational variants (e.g., G+V, G+R, G→V→R).

- **Generator (G):** Phi-2 (2.7B), LoRA-finetuned on GSM8K with structured output format.
- **Verifier (V):** RoBERTa-large binary classifier trained to predict correctness from full reasoning trace + answer.
- **Ranker (R):** RoBERTa-large regression model trained with margin-ranking loss to score solution quality.

Method	Selection Rule
Majority Vote	$a_{MV} = \arg \max_a \text{freq}(a)$
G+V (Top)	$a = a_i$ where $i = \arg \max_i V(s_i)$
G+V (Weighted)	$\text{Score}(a) = \sum_{i:a_i=a} V(s_i)$
G+R (Top)	$a = a_i$ where $i = \arg \max_i R(s_i)$
G+R (Weighted)	$\text{Score}(a) = \sum_{i:a_i=a} R(s_i)$
G→V→R (Top)	Apply V to filter (top- k), then pick highest R
G→V→R (Weighted)	Weighted sum of V and R scores

Table 1: Aggregation methods in SMOLSolver. $V(s_i) \in [0, 1]$ denotes the Verifier’s predicted probability of correctness, and $R(s_i)$ is the Ranker’s continuous quality score.

For each of the 1,319 GSM8K test problems, G produces 15 sampled solutions. Each solution $s_i = (c_i, a_i)$ consists of reasoning text c_i and extracted numeric answer a_i . V and R score each s_i , and final answers are selected via one of seven methods (Table 1).

3.2 Consensus Grouping

We compute answer frequencies across 15 samples:

$$r_1 = \max \text{freq}(a), \quad r_2 = \text{second-largest freq}(a),$$

$$\text{margin} = r_1 - r_2$$

Using thresholds $\tau_{\text{low}} = 0.133$, $\tau_{\text{high}} = 0.533$ (from validation), we partition problems into:

- **High-consensus:** $\text{margin} > \tau_{\text{high}}$
- **Medium-consensus:** $\tau_{\text{low}} < \text{margin} \leq \tau_{\text{high}}$
- **Low-consensus:** $\text{margin} \leq \tau_{\text{low}}$

This categorization allows us to analyze how different SMOLSolver configurations perform under varying levels of generator stability, from highly confident cases (high-consensus) to uncertain ones (low-consensus).

3.3 Aggregation Methods

We evaluate seven selection strategies (Table 1):

4 Experiments

4.1 Setup

We conduct experiments on the GSM8K dataset, which contains 8.5K linguistically diverse grade-school math word problems. Each problem requires 2 to 8 steps to solve using basic arithmetic operations. Solutions are expressed in natural language with a final numerical answer.

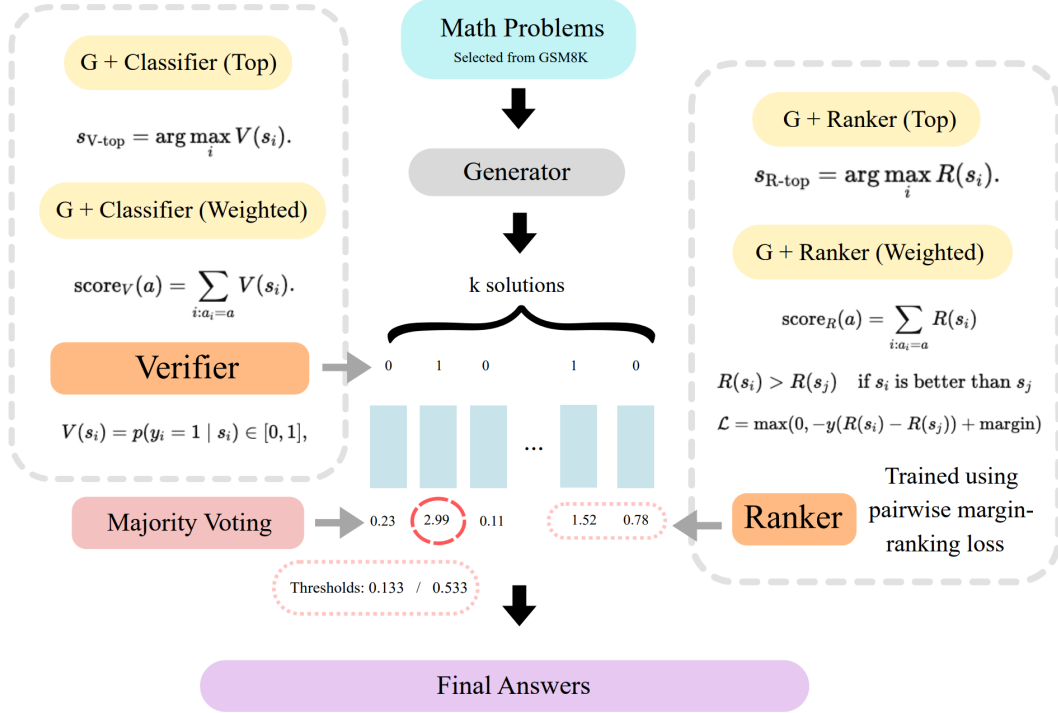


Figure 1: SMOLSolver pipeline. The Generator (Phi-2 with LoRA) produces multiple reasoning paths for each problem. The Verifier (binary classifier) and Ranker (regression model) score each candidate solution. A consensus-aware aggregation module then selects the final answer using methods such as majority voting or score-weighted selection.

Generator: Phi-2, LoRA (r=16), trained on GSM8K (2 epochs). On the GSM8K validation split, the model achieves 64.06% Pass@1.

Verifier & Ranker: RoBERTa-large, trained on synthetic data from Generator (15 samples $\times$ training set).

Test set: 1,319 GSM8K problems with 15 samples each (from `test_k15.json`). **Baselines:** Majority Vote, GPT-3.5 (57%), zero-shot Phi-2 ($\sim 20\%$)

Training Details: The Verifier and Ranker both use RoBERTa-large (355M params) with full fine-tuning, batch size 32, and learning rate $2e-5$. The Verifier is trained with cross-entropy loss (5 epochs, early stopping) on `train_k5.json`. The Ranker uses margin ranking loss (margin=0.5, 3 epochs) on pairwise samples from `train_k7.json`.

4.2 Main Results

Table 2 shows performance across consensus groups.

Key observations:

- Majority Vote is strong overall (68.61%), especially in the high-consensus group (96.04%).

- Ranker-weighted nearly matches MV overall (68.16%), and closely approaches MV in the medium-consensus subset (70.53% vs. 72.04%), while achieving the best accuracy in the low-consensus subset (32.37% vs. 32.13% for MV).
- Binary Verifier as a top selector fails badly (41.17%) — it lacks granularity.
- Multi-stage (G $\rightarrow$ V $\rightarrow$ R) degrades performance, suggesting redundancy.

Method	Overall	High	Medium	Low
Majority Vote	68.61%	96.04%	72.04%	32.13%
G+V (Top)	41.17%	65.15%	35.77%	17.27%
G+V (Weighted)	64.90%	95.05%	64.99%	28.30%
G+R (Top)	36.54%	59.41%	30.73%	14.39%
G+R (Weighted)	68.16%	95.84%	70.53%	32.37%
G $\rightarrow$ V $\rightarrow$ R (Top)	38.06%	60.79%	33.00%	15.35%
G $\rightarrow$ V $\rightarrow$ R (Weighted)	61.64%	93.07%	60.96%	24.22%

Table 2: Accuracy (%) across consensus groups.

4.3 Finding 1: Self-Critique Loop Degrades Performance

We attempted to improve generation quality using a **separately-trained TinyLLaMA-1.1B verifier**

(fine-tuned on 17,372 synthetic GSM8K examples with corrupted solutions) to critique and refine the Generator’s outputs. This self-critique approach aimed to identify and correct errors in the initial solutions. However, this method proved severely inadequate and **caused more harm than good**.

Results: On 300 evaluated examples, the self-critique system achieved only **34.7% accuracy** compared to the Generator’s baseline of **64.06%**—a catastrophic 29.36 percentage point collapse. The verifier:

- **Fixed** only 2 incorrect solutions (0→1)
- **Broke** 70 previously correct solutions (1→0)
- Yielded a fix-to-harm ratio of **0.029** (35× more harms than fixes)
- Rejected correct solutions at a 37% false positive rate
- Critiqued 63.3% of outputs—far too aggressively

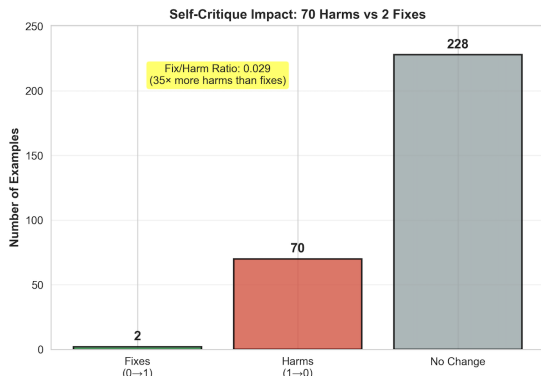


Figure 2: Self-critique catastrophic failure analysis across 300 evaluated examples.

Conclusion: The TinyLLaMA-1.1B verifier lacked the capacity to reliably distinguish correct from incorrect mathematical reasoning. Instead of refining solutions, it systematically degraded performance by incorrectly flagging valid answers. As shown in Figure 2, the verifier’s discrimination gap on synthetic training data (20.1%) did not translate to real error detection—highlighting a fundamental insufficiency in model scale and training approach for this verification task.

4.4 Finding 2: Increasing the Number of Generated Solutions Improves Generator Performance

To investigate the effect of sampling diversity on model reliability, we evaluate the LoRA-fine-tuned

Phi-2 Generator under varying numbers of sampled reasoning paths. For each problem in the GSM8K test set, the Generator produces k candidate solutions, and Pass@ k is used as the evaluation metric.

Results. Our results show a strong monotonic improvement in performance as the number of samples increases. Specifically, on the GSM8K test split, Pass@1 achieves only 45.56%, while larger sample sizes yield substantial gains: Pass@5 reaches approximately 60–76%, Pass@10 achieves 84.91%, and Pass@15 attains a peak accuracy of 88.55%. This corresponds to a +43% absolute improvement from $k = 1$ to $k = 15$. The trend is consistent across both the 1319-question test set and the 500-question evaluation split.

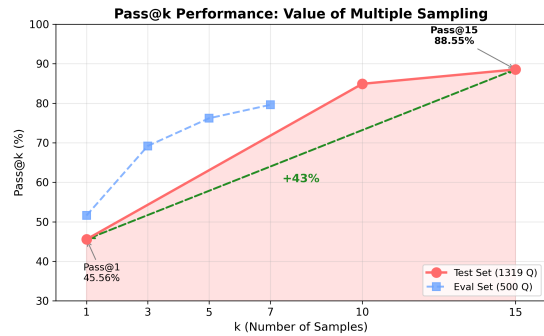


Figure 3: Pass@ k performance as a function of the number of sampled reasoning paths. The Generator benefits substantially from larger k , highlighting the value of multiple-sample generation.

Conclusion. The empirical results demonstrate that increasing the number of generated solutions markedly enhances the reliability of small language models on arithmetic reasoning tasks. Multi-sample generation provides a simple yet highly effective mechanism for improving performance without modifying model parameters.

4.5 Finding 3: Majority Vote Leads Overall, While Ranker-Weighted Excels in Medium- and Low-Consensus Settings

To study how aggregation strategies behave under different levels of generator stability, we partition the GSM8K test set into three consensus groups using answer-frequency margins: high-consensus (stable solution sets), medium-consensus (moderate variability), and low-consensus (high variability). We evaluate seven Generator–Verifier–Ranker aggregation methods across these subsets.

Results. Figure 4 shows that **Majority Vote (MV)** provides the strongest overall accuracy and performs particularly well on the high-consensus subset, achieving a peak of 96.04%. However, both the bar–line plot and the heatmap in Figure 5 reveal a more nuanced pattern in the unstable settings.

In the **medium-consensus** subset, where sampled reasoning paths are more diverse, the **Ranker-Weighted** method achieves 70.53%, outperforming all Verifier-based methods and closely matching MV. Notably, it is the only method that improves performance in this subset without degrading results elsewhere.

In the **low-consensus** subset—the most challenging regime with highly inconsistent generator outputs—**Ranker-Weighted achieves the highest accuracy (32.37%)**, surpassing MV (32.13%) and significantly outperforming all Verifier-based approaches (ranging from 17.3% to 28.3%). The Ranker’s continuous scoring function provides finer discrimination than the binary Verifier, making it particularly effective when the solution distribution is noisy or multi-modal.

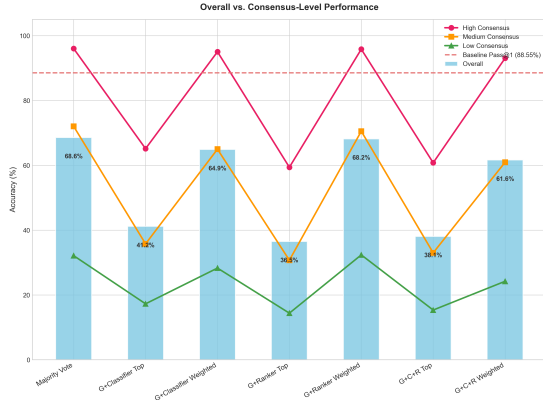


Figure 4: Overall vs. consensus-level performance across seven aggregation strategies. MV is strongest overall, while Ranker-Weighted uniquely improves performance on medium- and low-consensus subsets.

Conclusion. While Majority Vote remains the most robust aggregation method overall, the Ranker-Weighted strategy offers complementary benefits. It improves performance in unstable medium-consensus settings and achieves the strongest accuracy in the low-consensus subset, where voting-based methods are least reliable.

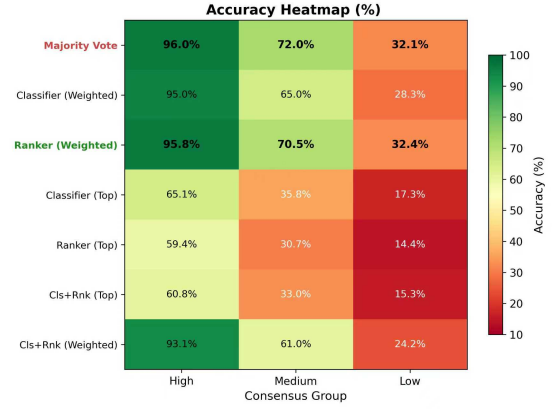


Figure 5: MV is strongest overall, while Ranker-Weighted provides the best performance among non-voting methods, and achieves the highest accuracy in the low-consensus subset.

4.6 Finding 4: Adaptive Sampling Achieves the Best Accuracy-Compute Trade-off

To explore whether computational resources can be allocated more efficiently than uniform sampling, we conduct a pilot study of an adaptive majority-voting strategy. The core idea is to increase the sampling budget (k) only when the generator shows high uncertainty, as indicated by a low consensus margin. In this proof-of-concept, we use an external model (Qwen2.5-Math-1.5B) as an oracle to decide when to expand from an initial $k = 5$ to a final $k = 10$ samples. The experiment is conducted on a subset of the GSM8K test set.

Results show that the adaptive method achieves a favorable accuracy–compute trade-off, recapturing a majority of the performance gap between fixed $k = 5$ and $k = 10$ sampling while using significantly fewer samples on average. Crucially, expansions are overwhelmingly beneficial or neutral: approximately **70%** of triggered expansions were neutral (leaving the final answer unchanged), while about **20%** successfully fixed an initially incorrect answer, as visualized in the breakdown shown in Figure 6. Expansions that altered the outcome fixed far more errors than they created. This pilot demonstrates that consensus-based adaptive sampling is a promising and efficient inference strategy for small models.

5 Discussion

Our investigation began with the hypothesis that a verifier could actively improve a generator’s output through step-level critique, creating a verifier-aided self-consistency loop. However, Finding 1

Outcomes When Expansion Occurs (Conditional Proportions)

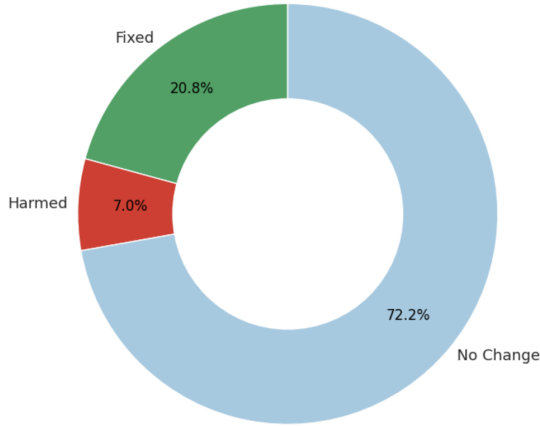


Figure 6: Breakdown of expansion outcomes in the adaptive sampling pilot. Approximately 70% of expansions were neutral (no change to answer), 20% successfully fixed an incorrect answer, and the remaining 10% represent cases where expansion altered a correct answer to an incorrect one.

revealed this paradigm fails at the scale of models we studied. The TinyLLaMA verifier, even when fine-tuned, lacked the robust mathematical reasoning capability required to reliably critique the generator’s step-by-step logic. Its feedback, often based on superficial patterns rather than deep correctness, degraded performance more often than it helped. This indicates that for the model class we examined, the active “verify-then-correct” cycle is too fragile, as an under-powered verifier itself becomes a significant source of error.

This failure led us to reframe the role of the auxiliary model from an *active critic* to a *passive scorer* within a post-hoc selection framework. Findings 2 and 3 demonstrate that this **verifier-aided generator self-consistency** approach is fundamentally more robust and scalable. The generator’s high Pass@15 rate reveals that its primary failure mode is stochastic inconsistency, not a fundamental lack of knowledge. The verifier—or, more effectively, a regression-based ranker—provides the necessary signal to identify the highest-quality solution among multiple varied samples. While simple Majority Voting exploits the generator’s self-consistency through frequency, a learned ranker enhances it by assessing semantic quality, proving particularly valuable in medium-consensus scenarios where voting is less decisive. This shifts the design philosophy from attempting to fix flawed

reasoning to intelligently selecting from a pool of potentially correct reasoning traces.

Looking forward, an **adaptive strategy for verifier-aided generator self-consistency** represents the most promising path for efficient and practical inference. As shown in Finding 4, the generator’s own uncertainty—proxied simply by the consensus margin among its samples—can dynamically guide the computational budget. For high-consensus problems where the generator is self-assured, a minimal sample size with basic voting may be sufficient. For problems exhibiting low consensus, the system can trigger the full verifier-aided pipeline: expanding the sample set and applying learned scoring or weighted aggregation. This creates a cost-effective, tiered system where the intensity of verification and sampling is matched to the difficulty perceived from the generator’s own output distribution, thereby optimizing the overall accuracy-compute trade-off without relying on internal model states.

Our work has several important limitations that outline clear avenues for future research. First, our evaluation is centered on the GSM8K dataset, which consists of short-form arithmetic problems. The effectiveness of this selection-over-correction paradigm for more complex, open-ended mathematical reasoning (e.g., competition-level problems or symbolic reasoning) remains an open question. Second, while our adaptive expansion pilot demonstrates feasibility, it relied on an external model as an oracle for the expansion decision. A self-contained, production-ready system would require developing and integrating a lightweight, trained module to predict uncertainty from the consensus signal, which adds design complexity. Third, all verification and ranking in our current system operate at the final solution level. Although process-level supervision is powerful for larger models, our initial results suggest step-level correction is highly challenging for the model class we studied. Finally, the generalizability of these specific patterns—especially the stark failure of corrective feedback and the relative gains from ranking—across other model architectures, sizes, and training regimes warrants further systematic study. The optimal balance between sampling, voting, and learned selection may shift with different model capabilities.

For practitioners, the implications are practical. When building reasoning systems with smaller models, investing in multi-sample generation and

robust selection mechanisms is essential. Attempting to construct real-time corrective feedback loops with auxiliary models of comparable scale is likely ineffective and risky. Instead, engineering effort should be directed towards improving candidate diversity, developing reliable scoring functions, and implementing efficient, uncertainty-aware sampling strategies that intelligently allocate more computation only where it is demonstrably needed.

6 Conclusion and Future Work

We presented SMOLSolver, a framework that explores effective paradigms for mathematical reasoning with small language models. Our key finding is that for this model class, **selection-based methods significantly outperform correction-based approaches**. An initial attempt to create a step-level iterative refinement loop failed, as a small verifier could not provide reliable corrective feedback. We successfully pivoted to a verifier-aided self-consistency paradigm, where generating diverse candidate solutions and then selecting the best one via consensus or a learned ranker proved robust. We further demonstrated that the generator’s own consensus is a powerful, low-cost signal of uncertainty, enabling adaptive strategies that improve compute efficiency.

This work suggests several productive directions for future research. First, the **adaptive expansion** concept should be matured into a fully learnable system. This involves training a lightweight router or gating mechanism to make the expansion decision directly from the consensus margin and other features, removing the need for an external oracle. Second, the **modular ranking** approach could be refined. Exploring whether a single model can perform both generation and ranking (e.g., through a unified training objective or a hidden-state scorer) would simplify the architecture. Third, the framework should be **tested on more complex reasoning benchmarks** to evaluate the boundaries of the selection-over-correction principle. Finally, a comprehensive study on how these dynamics change across a wider spectrum of model sizes and families would provide valuable guidance for the broader community building efficient reasoning systems.

In conclusion, for small language models, knowing *when to trust* the generator and *when to seek more evidence* is as crucial as the generation itself. We show that a lightweight, adaptive system built on selection and consensus can yield a favorable

balance between accuracy and computational cost, pointing toward more practical and efficient problem solving frameworks.

References

- Xinyu Guan, Li Lina Zhang, Yifei Liu, Ning Shang, Youran Sun, Yi Zhu, Fan Yang, and Mao Yang. 2025. [rstar-math: Small llms can master math reasoning with self-evolved deep thinking](#). *Preprint*, arXiv:2501.04519.
- Xiang Lisa Li, Vaishnavi Shrivastava, Siyan Li, Tatsunori Hashimoto, and Percy Liang. 2024. [Benchmarking and improving generator-validator consistency of language models](#). In *The Twelfth International Conference on Learning Representations*.
- Arindam Mitra, Hamed Khanpour, Corby Rosset, and Ahmed Awadallah. 2024. [Orca-math: Unlocking the potential of slms in grade school math](#). *Preprint*, arXiv:2402.14830.
- Juan Diego Rodriguez, Wenxuan Ding, Katrin Erk, and Greg Durrett. 2025. [Rankalign: A ranking view of the generator-validator gap in large language models](#). *Preprint*, arXiv:2504.11381.
- Yunxiang Zhang, Muhammad Khalifa, Lajanugen Logeswaran, Jaekyeom Kim, Moontae Lee, Honglak Lee, and Lu Wang. 2024. [Small language models need strong verifiers to self-correct reasoning](#). In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 15637–15653, Bangkok, Thailand. Association for Computational Linguistics.